



Unidad 5 Clase 10

Acceso a estructura NoSQL

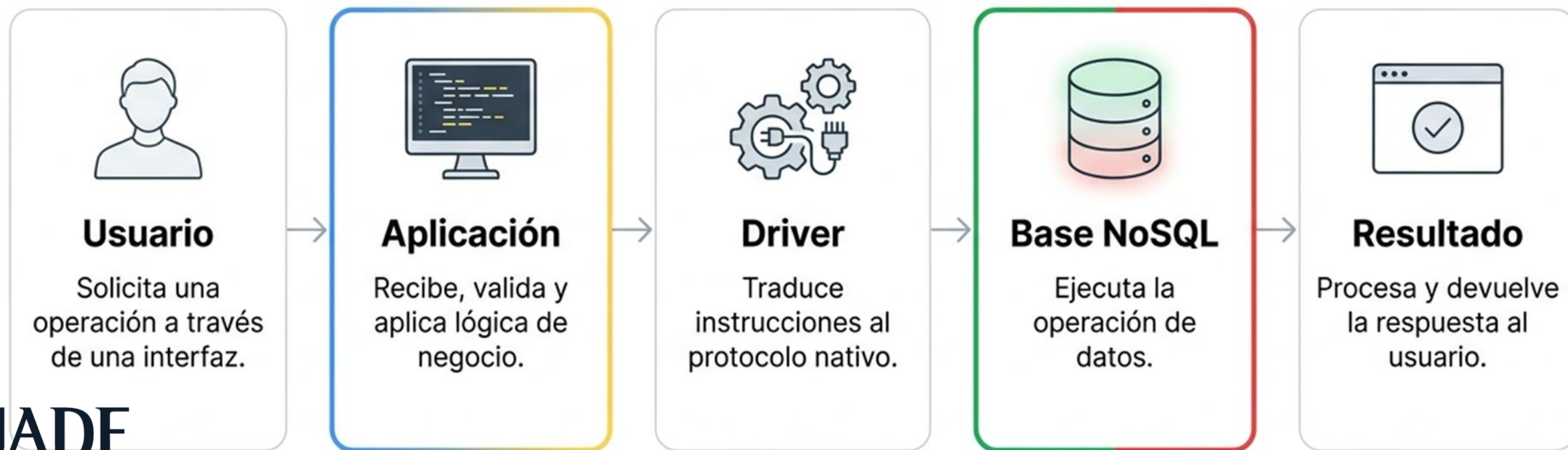
ACCESO A DATOS NOSQL DESDE APLICACIONES

Acceder y manipular datos almacenados en estructuras NoSQL desde aplicaciones, utilizando herramientas de conectividad y frameworks de persistencia, y evaluando los resultados obtenidos.

Acceso a datos NoSQL desde aplicaciones

Del uso aislado en consola a la integración en sistemas reales

Hasta ahora, la interacción con las bases de datos ocurría mediante comandos manuales en la consola. En escenarios reales, la base de datos adquiere verdadero valor cuando actúa como el motor de almacenamiento de una solución de software completa interactuando con usuarios a través de interfaces.



Por qué una base NoSQL necesita una aplicación

Motores como MongoDB o Redis no resuelven el problema de negocio por sí solos

La Base de Datos Aislada



- Solo almacena y recupera datos crudos.
- No comprende el contexto del negocio (ej. "¿Este paciente tiene deudas?").
- Acepta cualquier dato si el esquema lo permite (riesgo de inconsistencias).
- Carece de flujos de interacción con el usuario.

El Valor de la Aplicación



- **Valida:** Verifica formatos y lógica temporal antes de guardar.
- **Ejecuta reglas:** "Un turno no puede ser asignado si ya está ocupado."
- **Toma decisiones:** Evalúa saldos, bloqueos y flujos alternativos.
- **Transforma:** Convierte datos técnicos (BSON) en respuestas visuales o APIs.

De la consola al sistema real

Comparativa estructural: entorno de aprendizaje vs. entorno de producción



Característica	Consola (Comandos Directos)	Aplicación (Acceso Programático)
Operador	Ingeniero de datos o DBA.	Usuario final o sistema externo (API).
Interacción	Ejecución manual, instrucción por instrucción.	Flujo automatizado , secuencial y predefinido.
Protección	Nula. Alteración directa del estado de los datos.	Alta . Los datos pasan por validaciones de código.
Respuesta	Formato técnico crudo (JSON, logs en terminal).	Interpretada . Se formatea para consumo del cliente.
Manejo de Errores	Rompe la ejecución y muestra el error en pantalla.	Capturados (try/except) para ofrecer vías de recuperación.

Caso conductor: Sistema de turnos médicos

Aplicando persistencia polígloa a un escenario real



Una clínica necesita gestionar pacientes, médicos y horarios. Este modelo exige coordinar estados temporales rápidos (reservas en curso) con registros históricos permanentes (auditoría y catálogo).

Redis



Bloqueos temporales.

Cuando un paciente selecciona un turno, Redis lo bloquea (TTL de 5 min) para evitar concurrencia mientras confirma.

Aplicación Python



El director de orquesta.

Recibe la petición del paciente y ejecuta la lógica.

MongoDB



Catálogo e Historial.

Almacena pacientes, médicos y guarda la Reserva Confirmada de forma permanente.

Aplicación, Driver y Base de Datos

Anatomía de la conexión: Separando las responsabilidades de la arquitectura



Capa 1: La Aplicación (Lógica de Negocio)

Escrita en lenguajes como Python. Contiene las reglas: define qué hacer, cuándo y por qué. No sabe cómo guardar datos físicamente.



Capa 2: El Driver (Conexión y Traducción)

Librería intermedia instalada (ej. vía `pip`). Actúa como **traductor bidireccional** entre las estructuras del lenguaje (diccionarios) y los protocolos nativos de la base.



Capa 3: La Base de Datos (Motor de Almacenamiento)

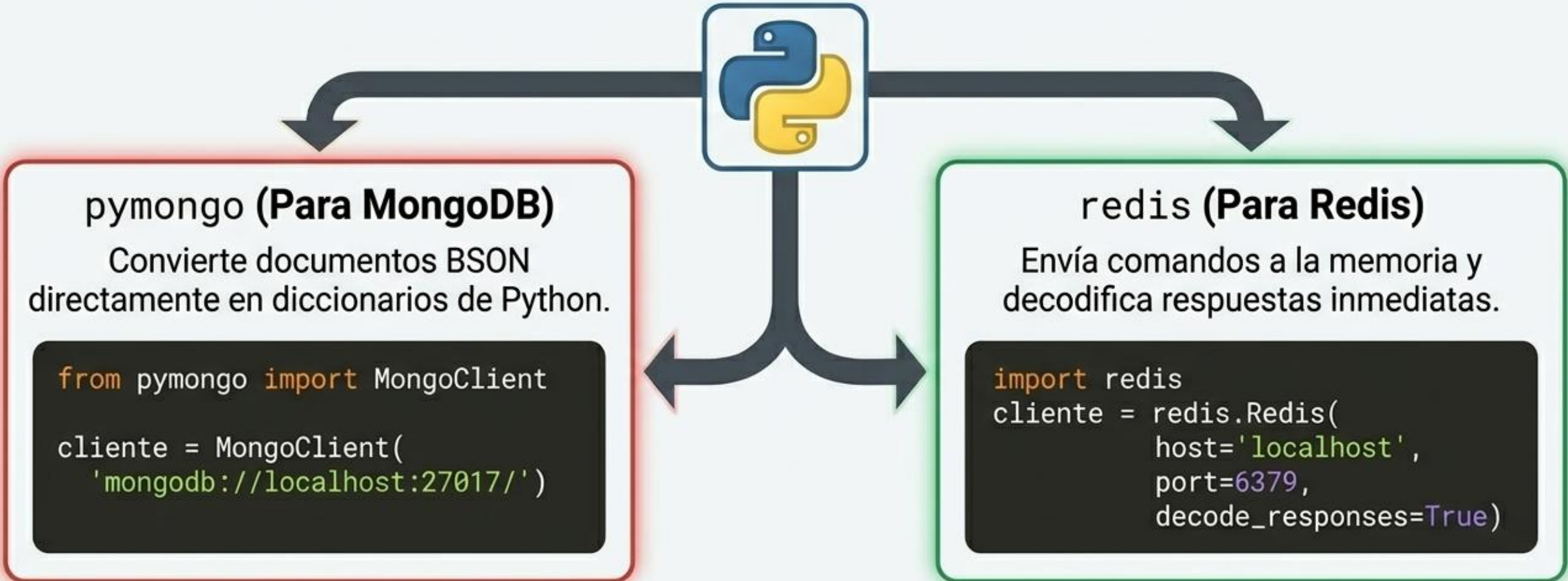
Recibe las operaciones ya traducidas, las ejecuta en disco o memoria, y devuelve una respuesta de éxito o fallo.

Drivers en Python para NoSQL

Python utiliza librerías especializadas para interactuar con los motores de datos



Python no 'habla' directamente con las bases; utiliza paquetes instalables mediante pip. Estos drivers exponen métodos idénticos a los comandos originales de las bases.



Operaciones CRUD desde una aplicación

Las 4 operaciones fundamentales aplicadas al ecosistema de turnos médicos



C - Create (Crear) `insert_one()`



Caso Médico: El paciente confirma la cita; la aplicación inserta un nuevo documento de 'Reserva' en MongoDB.

R - Read (Leer) `find()` / `find_one()`



Caso Médico: Consultar horarios disponibles para 'Cardiología' en la fecha '20-05-2026' para mostrarlos en la interfaz.

U - Update (Actualizar) `update_one()`



Caso Médico: Cambiar el campo 'estado' del turno de 'disponible' a 'confirmado' tras verificar el pago.

D - Delete (Eliminar) `delete_one()` /



Caso Médico: Cancelación de cita; se elimina el registro temporal en Redis y se libera el horario en MongoDB.

MongoDB como base documental

Estructuras de datos flexibles, pero con propósito lógico y disciplinado



```
{
  '_id': ObjectId('665a...'),
  'paciente_id': 'pac_1001',
  'medico_id': 'med_2001',
  'especialidad': 'Cardiología',
  'fecha': '2026-05-20',
  'hora': '10:30',
  'estado': 'confirmado'
}
```

Identificador inmutable generado automáticamente por MongoDB.

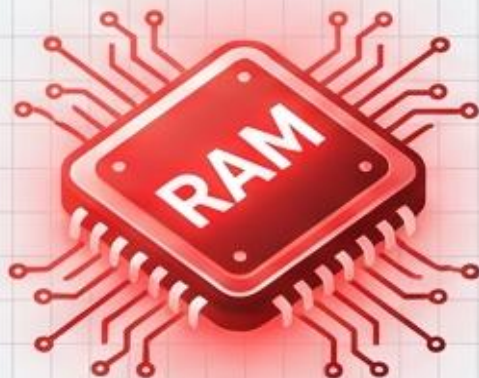
Relación referencial suave a la colección de pacientes.

Se representa nativamente en la aplicación como un Diccionario de Python.

! Flexibilidad no significa desorden

A diferencia de SQL, MongoDB no obliga a tener un esquema fijo (schema-less). Sin embargo, la aplicación Python es la responsable absoluta de asegurar que los turnos mantengan esta estructura lógica para garantizar búsquedas eficientes y coherencia total.

Redis: El motor clave-valor de alta velocidad



Definición: Base de datos en memoria (Remote Dictionary Server) orientada a la velocidad extrema y baja latencia.

Casos de uso clave: Caché, sesiones de usuario, contadores rápidos y bloqueos temporales distribuidos.

	 Redis	 MongoDB
Propósito Principal	Estado temporal y velocidad	Historial y persistencia documental
Medio de Almacenamiento	Memoria RAM	Disco (con caché)
Estructura de Datos	Clave-Valor, Listas, Sets	Documentos BSON flexibles
Rol en Caso Médico	Bloqueo momentáneo de un turno	Registro definitivo de la reserva

Persistencia vs. Estado Temporal en Arquitecturas



MongoDB: Datos que deben conservarse

- **Naturaleza:** Información histórica, trazabilidad y estado definitivo.
- **Aplicación:** Historial clínico del paciente, catálogo de médicos, reservas confirmadas.
- **Regla:** Sobrevive a reinicios del servidor. Es la fuente de la verdad a largo plazo.



Redis: Datos con valor por minutos

- **Naturaleza:** Información volátil, transiciones rápidas y control de concurrencia.
- **Aplicación:** Intentos de login fallidos, carritos de espera, y bloqueo temporal de un turno.
- **Regla:** Su desaparición no afecta el historial médico; simplemente libera recursos.

El patrón TTL (Time To Live): Evitando bloqueos indefinidos

Anatomía de una Clave Redis

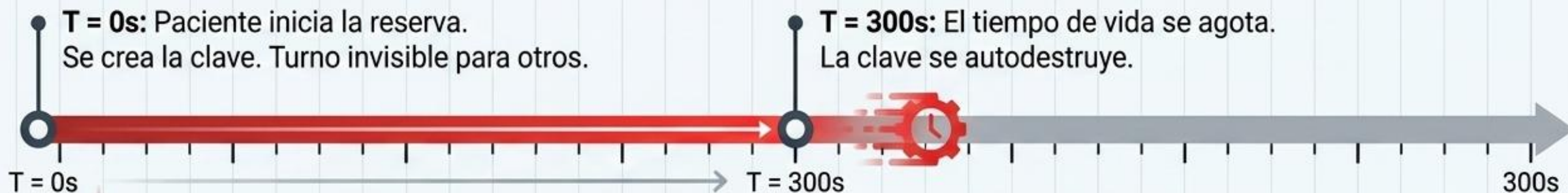
turno:bloqueado:med_2001:2026-05-20:10:30

[Namespace / Estado]

[ID del Profesional]

[Fecha y Hora específica]

El Reloj TTL



Conclusión: El TTL garantiza que si la aplicación falla, el recurso vuelve a estar disponible automáticamente, previniendo bloqueos infinitos.

Persistencia Políglota: La arquitectura en acción



Definición: La persistencia políglota es el uso coordinado de múltiples tecnologías de bases de datos en un mismo sistema, asignando a cada una la responsabilidad que mejor se ajusta a su diseño interno.

Preparación del entorno de trabajo

Workbench Toolkit

Orquestación y Desarrollo



Python: El lenguaje y motor de ejecución.

VS Code: El editor de código para organizar los scripts.

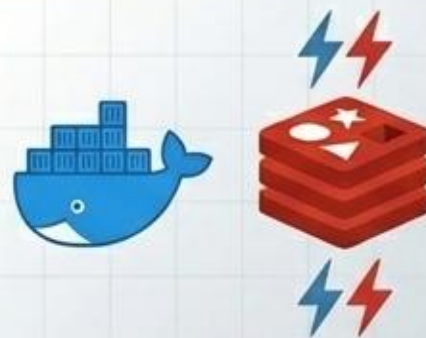
Persistencia



MongoDB: El motor documental local o en la nube.

MongoDB Compass: Interfaz gráfica para explorar documentos.

Estado Transitorio



Docker Desktop: Para ejecutar contenedores aislados.

Redis: Ejecutado vía Docker para gestionar bloqueos.

Drivers de Conexión



Librerías pymongo y redis: El puente de comunicación entre el código Python y las bases de datos.

Paso 1: Instalación base de Python

- ☒ Descargar instalador desde python.org (Versión 3.12+ recomendada).
-  **CRÍTICO:** Marcar la casilla 'Add Python to PATH' durante la instalación. Esto permite ejecutar Python desde cualquier ubicación.
- ☒ Completar la instalación estándar.

```
# Verificar la instalación de Python
> python --version
# (En algunos sistemas Windows usar: py --version)

# Verificar el gestor de paquetes pip
$ pip --version
# (Alternativa segura: python -m pip --version)
> 
```

Paso 2: Estructura del proyecto y entorno virtual

Árbol de Directorio

```
nosql_python_turnos/  
├── conexion_mongodb.py [Prueba  
│                       de conexión documental]  
├── conexion_redis.py [Prueba de  
│                    conexión en memoria]  
├── caso_turnos.py [Script  
│                 integrador del flujo médico]  
└── venv/ [Entorno virtual  
         aislado]
```

Comandos de Terminal

Crear el entorno virtual:

```
python -m venv venv
```

Activar el entorno (Windows):

```
.\venv\Scripts\activate
```

Activar el entorno (Mac/Linux):

```
source venv/bin/activate
```

❗ Nota: Un prefijo (**venv**) aparecerá en la terminal confirmando que el entorno está activo.

Paso 3: Instalación de drivers y el "Entorno Burbuja"

```
# Conector para MongoDB
pip install pymongo
Collecting pymongo:
  Ready already in maste-0-1.205/ install pymongo 1.0.2
Already install maste-0-1.205

# Conector para Redis
pip install redis
Collecting redis:
  Ready already in last-path 1.0.1

# (Opcional) Para variables de entorno
pip install python-dotenv
Collecting dotenv:
  Ready already :python-dotenv/ install python-dotenv

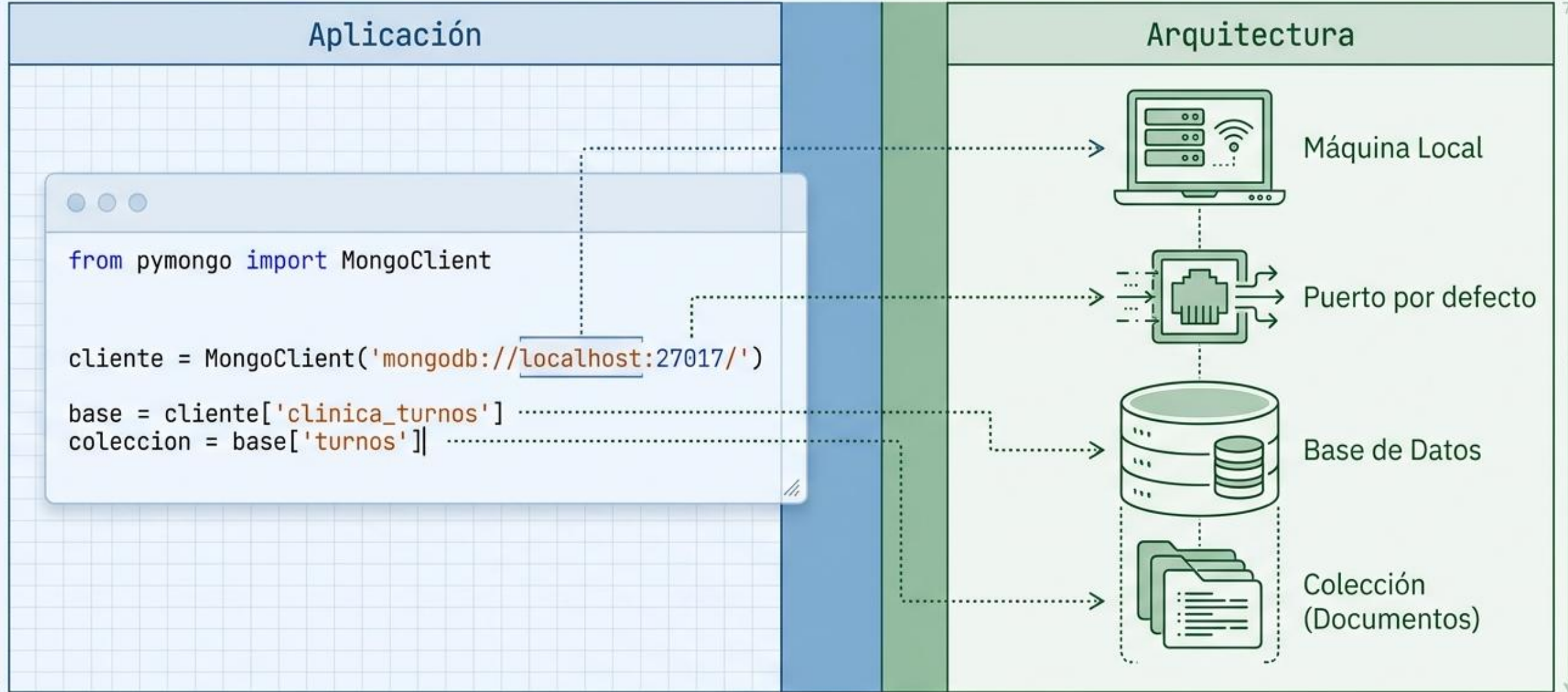
# Verificar librerías instaladas
pip list
Package      Version
-----
pymongo      10.2.1
redis        11.0.8
```



¿Por qué usar Entornos Virtuales?

- **Aislamiento:** Crea una burbuja protectora. Las librerías instaladas aquí no afectan a otros proyectos.
- **Reproducibilidad:** Asegura que el proyecto use las versiones exactas que necesita, evitando conflictos.
- **El Rol de los Drivers:** Son los traductores oficiales. Sin ellos, Python no sabría qué idioma hablar con las bases de datos.

Conexión a MongoDB: El Puente de Persistencia

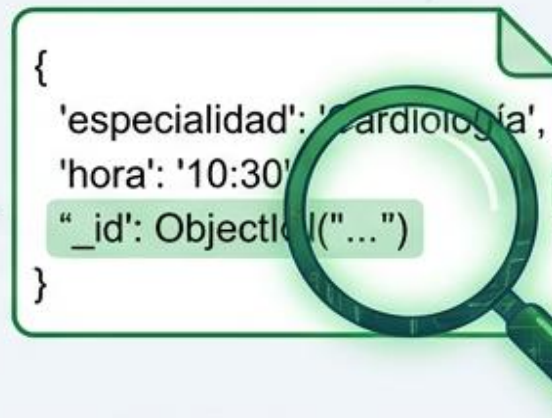


Nota: Si la base o colección no existen, MongoDB las crea automáticamente al insertar el primer documento.

Operaciones Básicas: Inserción y Consulta

Acción: Inserción

```
turno = {'especialidad': 'Cardiología', 'hora': '10:30'}  
resultado = coleccion.insert_one(turno)
```



MongoDB genera un identificador BSON único automáticamente.

Acción: Consulta

```
turnos = coleccion.find({'especialidad': 'Cardiología'})  
for t in turnos:  
    print(t)
```



{'especialidad': 'Cardiología'}



Ciclo For (Cursor)

Trazabilidad: Cancelación Lógica vs. Eliminación Física

Solución Recomendada

matched_count:

Verifica si el documento original existe.

modified_count:

Confirma si el cambio de estado fue efectivo.

```
resultado = coleccion.update_one(
    filtro, {'$set': {'estado': 'cancelado'}}
)
```

Criterio	Eliminación Física (delete_one)	Cancelación Lógica (update_one - RECOMENDADO)
Impacto en auditoría	Pérdida total del historial	Trazabilidad preservada
Uso en sistemas críticos	Evitado rigurosamente	Práctica estándar recomendada
Manejo de errores	Irreversible	Reversible

Cancelar un turno cambiando su estado conserva la historia clínica; borrarlo destruye el contexto.

Despliegue de Redis: Infraestructura Efímera con Docker

Asigna un identificador único al contenedor.

Mapea el puerto local al puerto interno del contenedor.

```
docker run --name redis-turnos -p 6379:6379 -d redis
```

Detached mode (ejecución silenciosa en segundo plano).

Imagen oficial a descargar del registro de Docker.

Cheatsheet Operativo



docker ps

Verificar servicios activos



docker stop redis-turnos

Detener el contenedor



docker start redis-turnos

Reiniciar el servicio

Conexión a Redis: La Capa de Baja Latencia

Aplicación

```
import redis

cliente_redis = redis.Redis(
    host='localhost', port=6379, decode_responses=True
)

cliente_redis.ping()
```

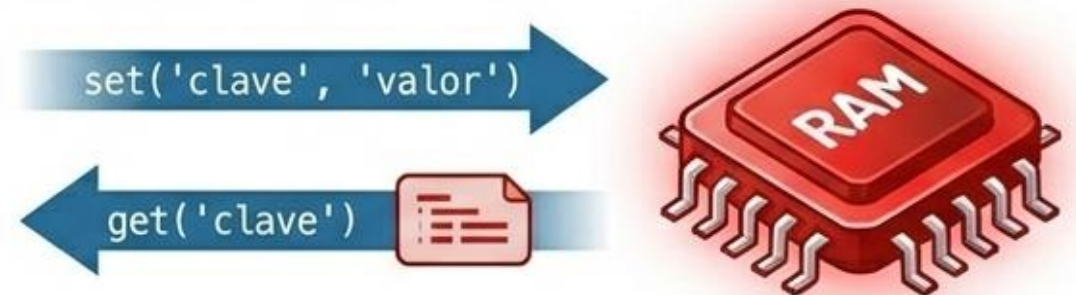
Convierte automáticamente los *bytes* a *strings* legibles en Python.

Arquitectura de Ejecución

El Pulso de Verificación

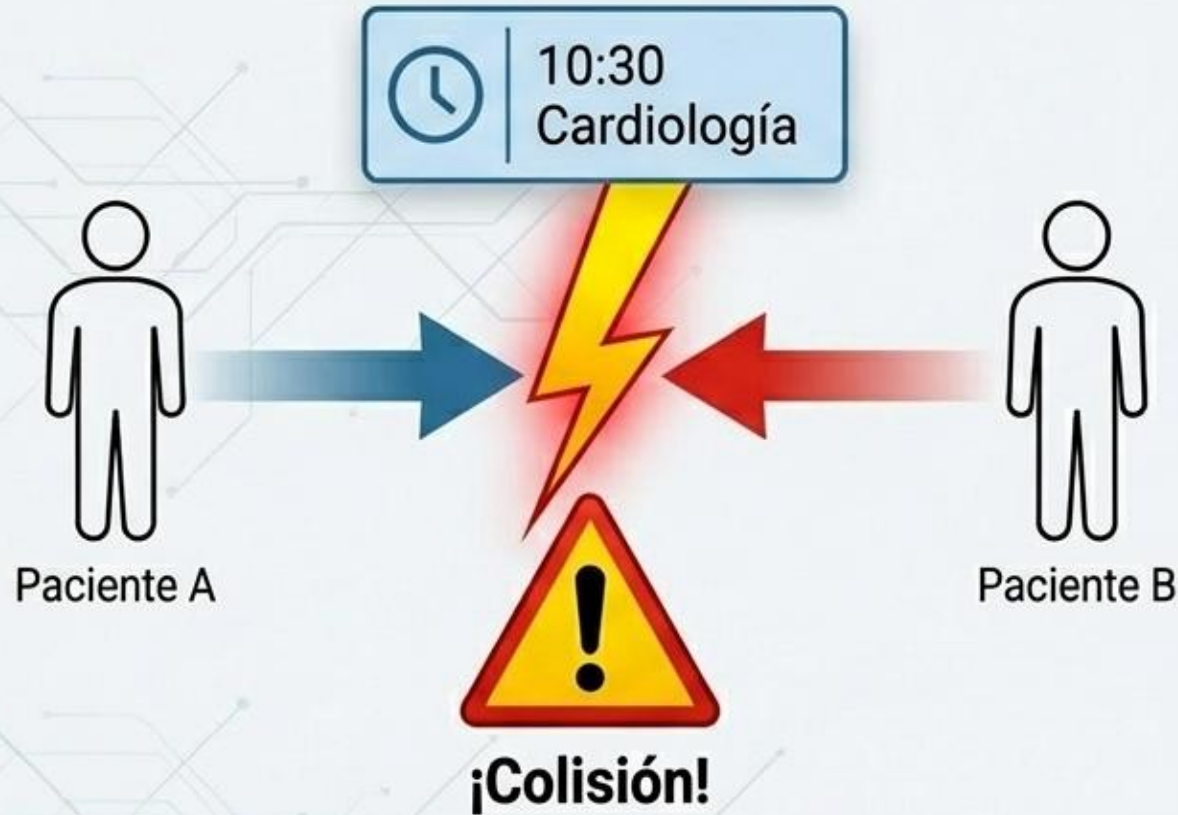


Estructura Clave-Valor



Control de Concurrency: Prevención de Colisiones

El Problema: Condición de Carrera



La Solución: Bloqueo en Redis

```
bloqueo = cliente_redis.set(clave,  
                             paciente_id, ex=300, nx=True)
```

Anatomía del Bloqueo

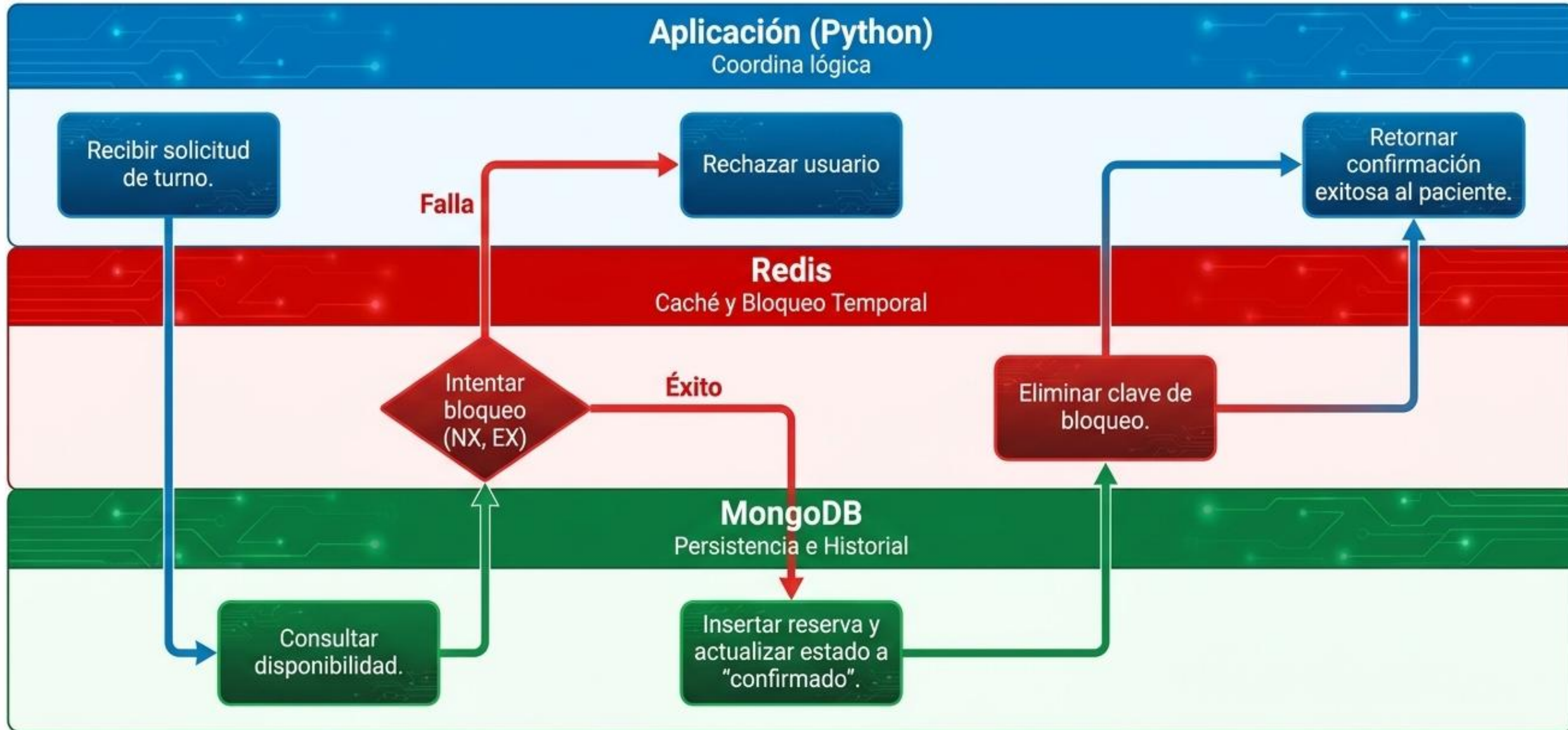


Set IF NOT EXISTS.
Redis crea la clave SOLO si no existe, bloqueando efectivamente al segundo paciente.

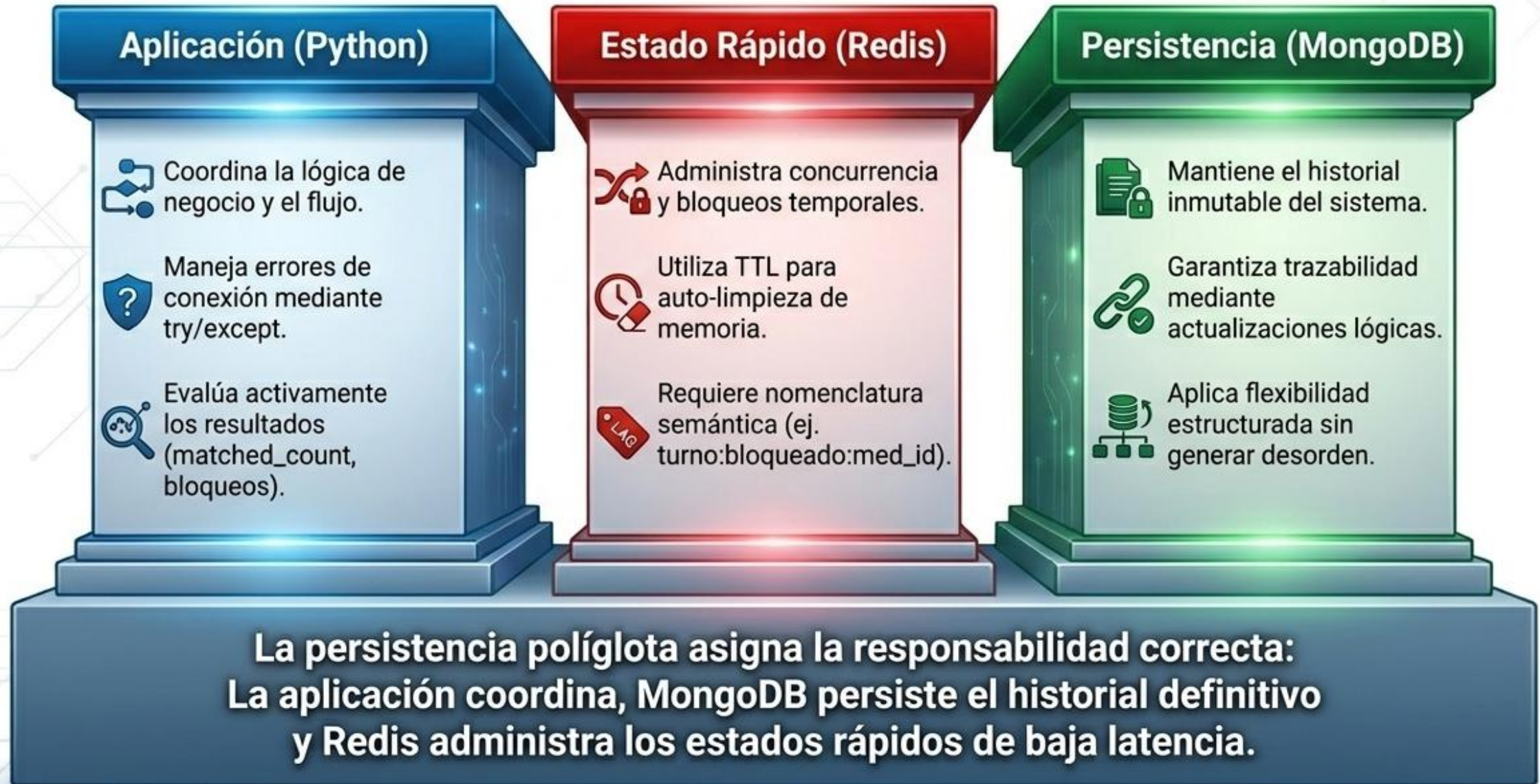


Time-To-Live (TTL)
Tiempo de vida de 5 minutos (300s). Previene bloqueos indefinidos si el usuario abandona la página.

Orquestación del Sistema: Flujo Completo de Reserva



Arquitectura de Datos: Síntesis y Buenas Prácticas





Práctica de laboratorio:

Gestión de turnos médicos con MongoDB y Redis

Una clínica médica necesita implementar un sistema simple de reserva de turnos. El sistema debe permitir consultar turnos disponibles, reservar un turno para un paciente y evitar que dos pacientes reserven el mismo horario al mismo tiempo.

Para resolver esta necesidad se utilizarán dos bases NoSQL con responsabilidades diferentes:

- MongoDB almacenará la información persistente: médicos, turnos y reservas confirmadas.
- Redis administrará bloqueos temporales para evitar conflictos cuando dos pacientes intentan reservar el mismo turno.

La aplicación será desarrollada en Python. Python actuará como capa de integración entre el usuario, MongoDB y Redis.



PROXIMA CLASE



Clase 11 - Arquitecturas híbridas y persistencia polimórfica

Tema:

Persistencia polimórfica.

Acceso a múltiples fuentes de datos.

Diseño de soluciones híbridas (SQL + NoSQL).
